

Reference

pyradiance

pyradiance is a Python interface for Radiance. It is a collection of Python modules that provide a high level interface to Radiance. It is designed to make Radiance easier to use and accessible to Python user.

Pcomb

```
__init__(xres=None, yres=None, inform='a', fout=True, header=False, expression=None, source=None)
```

combine Radiance pictures and/or float matrices

Parameters:

Name	Type	Description	Default
xres	None int	horizontal resolution	None
yres	None int	vertical resolution	None
inform	str	input data format. Default is "a" for ascii.	'a'
fout	bool	if True, write output to file	True
header	bool	if True, write header	False
expression	None str	expression	None
source	None str	source	None

```
add(image, original=False, scaler=1.0)
```

Add images to command.

Parameters:

Name	Type	Description	Default
image	Path str bytes	Input image file or bytes	<i>required</i>
original	bool	keep original exposure	False
scaler	float	Scaling factor	1.0

Returns:

Type	Description
Pcomb	self

Primitive dataclass

Radiance Primitive.

Attributes one-to-one mapped from Radiance.

Attributes:

Name	Type	Description
modifier	str	modifier, which primitive modifies this one
ptype	str	primitive type
identifier	str	identifier, name of this primitive
sargs	Sequence[str]	string arguments
fargs	Sequence[float]	real arguments

RayParams

```
__module__ = 'pyradiance.rad_params' class
```

`str(object="") -> str` `str(bytes_or_buffer[, encoding[, errors]]) -> str`

Create a new string object from the given object. If encoding or errors is specified, then the object must expose a data buffer that will be decoded using the given encoding and error handler. Otherwise, returns the result of `object.str()` (if defined) or `repr(object)`. encoding defaults to `sys.getdefaultencoding()`. errors defaults to 'strict'.

aa property

ambient accuracy

ab property

ambient bounce

ad property

ambient division

af property

ambient file

ambincl property

ambient inclusion/exclusion

amblist property

(self) -> list

ar property

ambient resolution

as_ property

ambient super sampling

av property

ambient values

aw property

ambient weight

bv property

source back side visibility

dc property

direct certainty

dj property

direct jitter

dp property

virtual source pretesting

dr property

direct relay

ds property

source size aspect ratio

dt property

direct threshold

dv property

direct visibility

i property

do irradiance

lr property

max depth

lw property

minimum ray weight

ma property

medium (mist) scattering albedo

me property

medium(mist) extinction coefficients

mg property

medium (mist) eccentricity factor

ms property

medium (mist) sampling distance

ss property

specular jitter

st property

specular threshold

u property

random sampling

__init__(*args, **kwargs) method descriptor

Initialize self. See help(type(self)) for accurate signature.

__new__(*args, **kwargs) builtin

Create and return a new object. See help(type) for accurate signature.

Rcomb

__init__(transform=None, transform_all=None, source=None, expression=None, concat=None, outform=None, header=True, silent=False)

Combine multiple rasters.

Parameters:

Name	Type	Description	Default
<code>transform</code>	<code>None str</code>	transform	<code>None</code>
<code>transform_all</code>	<code>None str</code>	transform all	<code>None</code>
<code>source</code>	<code>None str</code>	source	<code>None</code>
<code>expression</code>	<code>None str</code>	expression	<code>None</code>
<code>concat</code>	<code>None Sequence[str]</code>	concat	<code>None</code>
<code>outform</code>	<code>None str</code>	output format	<code>None</code>
<code>header</code>	<code>bool</code>	include header	<code>True</code>
<code>silent</code>	<code>bool</code>	suppress output	<code>False</code>

`add_input(input, transform=None, scale=None)`

Insert commands for inputs

Parameters:

Name	Type	Description	Default
<code>input</code>	<code>str Path bytes</code>	input can be file path or bytes	<i>required</i>
<code>transform</code>	<code>None str</code>	transform string	<code>None</code>
<code>scale</code>	<code>None Sequence[float]</code>	sequence of scaling factors	<code>None</code>

Returns:

Type	Description
<code>Rcomb</code>	self



Resolu

`__module__ = 'pyradiance.rad_params'` class

`str(object="") -> str str(bytes_or_buffer[, encoding[, errors]]) -> str`

Create a new string object from the given object. If encoding or errors is specified, then the object must expose a data buffer that will be decoded using the given encoding and error handler. Otherwise, returns the result of `object.str()` (if defined) or `repr(object)`. encoding defaults to `sys.getdefaultencoding()`. errors defaults to 'strict'.

`rt` property

`(self) -> int`

`xr` property

`(self) -> int`

`yr` property

`(self) -> int`

`__init__()` method descriptor

`init(self) -> None`

`__new__(*args, **kwargs)` builtin

Create and return a new object. See `help(type)` for accurate signature.

Scene

Radiance Scene.

`materials` property

Scene materials.

`octree` property

Scene id.

`sid` property

Scene id.

`sources` property

Scene sources.

`surfaces` property

Scene surfaces.

`__init__(sid, **kwargs)`

Parameters:

Name	Type	Description	Default
<code>sid</code>	<code>str</code>	scene id	<i>required</i>

`add_material(material)`

Add material to the scene. Args: material: material to be added

`add_sensor(sensor)`

Add sensor to the scene. Args: sensor: sensor to be added

`add_source(source)`

Add source to the scene. Args: source: source to be added

`add_surface(surface)`

Add surface to the scene. Args: surface: surface to be added

`add_view(view)`

Add view to the scene. Args: view: view to be added

`build()`

Build an octree, as {sid}.oct in the current directory. Will not build if scene has not changed since last build.

`remove_material(material)`

Remove material from the scene. Args: material: material to be removed

`remove_source(source)`

Remove source from the scene. Args: source: source to be removed

`remove_surface(surface)`

Remove surface from the scene. Args: surface: surface to be removed

`View`

`__module__ = 'pyradiance.rad_params'` `class`

`str(object="") -> str` `str(bytes_or_buffer[, encoding[, errors]]) -> str`

Create a new string object from the given object. If encoding or errors is specified, then the object must expose a data buffer that will be decoded using the given encoding and error handler. Otherwise, returns the result of `object.str()` (if defined) or `repr(object)`. encoding defaults to `sys.getdefaultencoding()`. errors defaults to 'strict'.

`hn2` `property`

`(self) -> float`

`hoff` `property`

View horizontal offset

`horiz` `property`

View horizontal size

`hvec` `property`

`(self) -> tuple`

`type` `property`

View types: 'v': perspective, 'l': parallel, 'c': cylindrical panorma, 'h': hemispherical fisheye, 'a': angular fisheye, 's': planispherical fisheye

`vaft` `property`

View aft clipping plane

`vdir` `property`

View direction: x, y, z

vdist property

(self) -> float

vert property

View vertical size

vfore property

View fore clipping plane

vn2 property

(self) -> float

voff property

View vertical offset

vp property

View position: x, y, z

vu property

View up direction: x, y, z

vvec property

(self) -> tuple

__init__() method descriptor

init(self) -> None

__new__(*args, **kwargs) builtin

Create and return a new object. See help(type) for accurate signature.

WrapBSDF

__init__(inxml=None, enforce_window=False, comment=None, correct_solid_angle=False, basis=None, unlink=False, unit=None, geometry=None, **kwargs)

Initialize wrapper operation of BSDF data into XML

Parameters:

Name	Type	Description	Default
<code>inxml</code>	<code>None str Path</code>	input xml file	<code>None</code>
<code>enforce_window</code>	<code>bool</code>	Enforcing LBNL Window XML schema,	<code>False</code>
<code>comment</code>	<code>None str</code>	additional comment to add to XML,	<code>None</code>
<code>correct_solid_angle</code>	<code>bool</code>	Correct the input BSDF by solid angles,	<code>False</code>
<code>basis</code>	<code>None str</code>	BSDF basis to use, "kf", "kh", "kq",	<code>None</code>
<code>unlink</code>	<code>bool</code>	Whether to remove input file after creating XML file,	<code>False</code>
<code>unit</code>	<code>None str</code>	BSDF geometry unit,	<code>None</code>
<code>geometry</code>	<code>None str</code>	Whether to include geometry in XML,	<code>None</code>
<code>**kwargs</code>	<code>str float</code>	Additional tags to be passed into XML	<code>{}</code>

`add_solar(tb=None, tf=None, rb=None, rf=None)`

Insert commands for solar data

Parameters:

Name	Type	Description	Default
<code>tb</code>	<code>None str</code>	back transmittance file	<code>None</code>
<code>tf</code>	<code>None str</code>	front transmittance file	<code>None</code>
<code>rb</code>	<code>None str</code>	back reflectance file	<code>None</code>
<code>rf</code>	<code>None str</code>	front reflectance file	<code>None</code>

Returns:

Type	Description
<code>WrapBSDF</code>	self

```
add_visible(tb=None, tf=None, rb=None, rf=None)
```

Insert commands for visible data

Parameters:

Name	Type	Description	Default
<code>tb</code>	<code>None str</code>	back transmittance file	<code>None</code>
<code>tf</code>	<code>None str</code>	front transmittance file	<code>None</code>
<code>rb</code>	<code>None str</code>	back reflectance file	<code>None</code>
<code>rf</code>	<code>None str</code>	front reflectance file	<code>None</code>

Returns:

Type	Description
<code>WrapBSDF</code>	self

Xform

```
__init__(inp, expand_cmd=True, invert=False, iprefix=None, modifier=None)
```

Initialize a transformation operation of a RADIANCE scene description

Parameters:

Name	Type	Description	Default
<code>inp</code>	<code>str</code> <code>Path</code> <code>bytes</code>	Input file or bytes	<i>required</i>
<code>expand_cmd</code>	<code>bool</code>	Set to True to expand command	<code>True</code>
<code>iprefix</code>	<code>None</code> <code>str</code>	Prefix identifier	<code>None</code>
<code>modifier</code>	<code>None</code> <code>str</code>	Set surface modifier to this name	<code>None</code>
<code>invert</code>	<code>bool</code>	Invert surface normal	<code>False</code>

```
array(number)
```

Insert array command.

Parameters:

Name	Type	Description	Default
<code>number</code>	<code>int</code>	array number	<i>required</i>

Returns:

Type	Description
Xform	self

`iterate(number)`

Insert iterate command.

Parameters:

Name	Type	Description	Default
number	int	iterate number	<i>required</i>

Returns:

Type	Description
Xform	self

`mirrorx()`

Insert mirror about yz plane command.

Returns:

Type	Description
Xform	self

`mirrory()`

Insert mirror about xz plane command.

Returns:

Type	Description
Xform	self

mirrorz()

Insert mirror about xy plane command.

Returns:

Type	Description
Xform	self

rotatex(deg)

Insert rotation around x axis command.

Parameters:

Name	Type	Description	Default
deg	float	rotation in degree	<i>required</i>

Returns:

Type	Description
Xform	self

rotatey(deg)

Insert rotation around x axis command.

Parameters:

Name	Type	Description	Default
<code>deg</code>	<code>float</code>	rotation in degree	<i>required</i>

Returns:

Type	Description
<code>Xform</code>	self

`rotatez(deg)`

Insert rotation around x axis command.

Parameters:

Name	Type	Description	Default
<code>deg</code>	<code>float</code>	rotation in degree	<i>required</i>

Returns:

Type	Description
<code>Xform</code>	self

`scale(ratio)`

Insert scaling command.

Parameters:

Name	Type	Description	Default
<code>ratio</code>	<code>float</code>	scaling factor	<i>required</i>

Returns:

Type	Description
<code>Xform</code>	self

`translate(x, y, z)`

Insert translate command.

Parameters:

Name	Type	Description	Default
<code>x</code>	<code>float</code>	translation in x coordinate	<i>required</i>
<code>y</code>	<code>float</code>	translation in y coordinate	<i>required</i>
<code>z</code>	<code>float</code>	translation in z coordinate	<i>required</i>

Returns:

Type	Description
<code>Xform</code>	self

`bsdf2klems(*inp, spp=None, half=False, quater=False, progress_bar=False, progress_bar_length=None, maxlobes=None, forward=False, backward=True, expr=None, file=None)`

Generate XML Klems matrix description of a BSDF.

Parameters:

Name	Type	Description	Default
<code>inp</code>	<code>str</code>	Path to XML file	<code>()</code>

Name	Type	Description	Default
spp	None int	number of samples for each input-output patch pair, default=1024.	None
half	bool	Generate instead a half Klems basis XML.	False
quater	bool	Generate instead a quater Klems basis XML.	False
progress_bar	bool	toggle to show progress bar.	False
progress_bar_length	None int	length of the progress bar, default=79 characters.	None
maxlobes	None int	maximum number of lobes in any interpolated radial basis function (default=15000). Setting the value to 0 turns off this limit.	None
forward	bool	generate forward matrix (default=off).	False
backward	bool	generate backward matrix (default=on).	True
expr	None str	expression to evaluate.	None
file	None str	file to write the output to	None

Returns:

Type	Description
bytes	The output of the command

```
bsdf2ttree(*inp, isotropic=False, reciprocity_averaging=True,
resolution=6, percent_cull=[90], super_samples=256,
difference_threshold=0.35, progress_bar=False,
progress_bar_length=None, maxlobes=15000, forward=False,
backward=True, expr=None, file=None)
```

Generate XML tensor tree description of a BSDF.

Parameters:

Name	Type	Description	Default
inp	str	Path to XML file	()
isotropic	bool	Generate an isotropic tree.	False
reciprocity_averaging	bool	Use reciprocity averaging.	True
resolution	int	resolution of the tree.	6
percent_cull	Sequence[float]	percent of the tree to cull.	[90]
super_samples	int	number of samples for each input-output patch pair, default=1024.	256
difference_threshold	float	difference threshold for culling.	0.35
progress_bar	bool	toggle to show progress bar.	False

Name	Type	Description	Default
<code>progress_bar_length</code>	<code>None int</code>	length of the progress bar, default=79 characters.	<code>None</code>
<code>maxlobes</code>	<code>int</code>	maximum number of lobes in any interpolated radial basis function (default=15000). Setting the value to 0 turns off this limit.	<code>15000</code>
<code>forward</code>	<code>bool</code>	generate forward matrix (default=off).	<code>False</code>
<code>backward</code>	<code>bool</code>	generate backward matrix (default=on).	<code>True</code>
<code>expr</code>	<code>None str</code>	expression to evaluate.	<code>None</code>
<code>file</code>	<code>None str</code>	file to write the output to	<code>None</code>

Returns:

Type	Description
<code>bytes</code>	Tensor tree BSDF XML in bytes

`cnt(*dims, shuffled=False)`

Index counter.

Parameters:

Name	Type	Description	Default
<code>dims</code>	<code>int</code>	list of dimensions	<code>()</code>

Name	Type	Description	Default
<code>shuffled</code>	<code>bool</code>	if True, the output will be shuffled	<code>False</code>

Returns:

Name	Type	Description
<code>bytes</code>	<code>bytes</code>	output of cnt

Examples:

```
>>> cnt(2, 3)
b'0 0\n0 1\n0 2\n1 0\n1 1\n1 2\n'
>>> cnt(2, 3, shuffled=True)
b'1 2\n0 1\n1 0\n0 2\n1 1\n0 0\n'
```

```
create_default_view(*args, **kwargs)
```

`create_default_view()` -> `pyradiance.rad_params.View`

```
dctimestep(*mtx, nstep=None, header=True, xres=None, yres=None,
inform=None, outform=None, ospec=None)
```

Call `dctimestep` to perform matrix multiplication.

Parameters:

Name	Type	Description	Default
<code>mtx</code>	<code>str</code> <code>bytes</code>	input matrices	<code>()</code>
<code>nstep</code>	<code>None</code> <code>int</code>	number of steps	<code>None</code>
<code>header</code>	<code>bool</code>	include header	<code>True</code>
<code>xres</code>	<code>None</code> <code>int</code>	x resolution	<code>None</code>

Name	Type	Description	Default
yres	None int	y resolution	None
inform	None str	input format	None
outform	None str	output format	None
ospec	None str	output specification	None

Returns:

Name	Type	Description
bytes	None bytes	output of dctimestep

```
evalglare(inp, view=None, detailed=False, ev_only=False, ev=None,
smooth=False, threshold=None, task_area=None, masking_file=None,
band_lum_angle=None, check_file=None, correction_mode=None,
peak_extraction=True, peak_extraction_value=50000, bg_lum_mode=0,
search_radius=0.2, version=False, source_color=None, fast=None)
```

Run evalglare on a Radiance image.

Parameters:

Name	Type	Description	Default
inp	str bytes Path	input image	<i>required</i>
view	None list[str]	view parameters	None
detailed	bool	detailed output	False

Name	Type	Description	Default
ev_only	bool	return vertical illuminance value	False
ev	None float	vertical illuminance value to use instead of the one computer from the image.	None
smooth	bool	enable smoothing function.	False
threshold	None float	Threshold factor.	None
task_area	None tuple	task area	None
masking_file	None str Path	masking file	None
band_lum_angle	None float	band luminance angle	None
check_file	None str Path	check file path.	None
correction_mode	None str	correction mode	None
peak_extraction	bool	enable luminance peak extraction	True
peak_extraction_value	float	luminance peak extraction value	50000
bg_lum_mode	int	background luminance calculation mode	0
search_radius	float	search radius	0.2

Name	Type	Description	Default
<code>version</code>	<code>bool</code>	print version	<code>False</code>
<code>source_color</code>	<code>None</code> <code>tuple[float, float, float]</code>	source color	<code>None</code>

Returns:

Type	Description
<code>bytes</code>	Evalglare output

```
falsecolor(inp, pic_overlay=None, contour=None, extrema=False,
scale=None, digits=None, label=None, ndivs=None, lwidth=None,
lheight=None, decades=None, multiplier=None, palette=None, redv=None,
grnv=None, bluv=None)
```

Generate a falsecolor Radiance picture.

Parameters:

Name	Type	Description	Default
<code>inp</code>	<code>str</code> <code>Path</code> <code>bytes</code>	Path or bytes to input picture file.	<i>required</i>
<code>pic_overlay</code>	<code>None</code> <code>str</code>	Path to another picture to overlay with contours.	<code>None</code>
<code>contour</code>	<code>None</code> <code>str</code>	Type of contour ('b' for bands, 'l' for lines, 'p' for posterization).	<code>None</code>
<code>extrema</code>	<code>bool</code>	Flag to print extrema points on the brightest and darkest pixels.	<code>False</code>

Name	Type	Description	Default
<code>scale</code>	<code>None str</code>	Scale for the false color (e.g., 'auto' or specific scale).	<code>None</code>
<code>digits</code>	<code>None int</code>	Max number of decimal places for legend entries.	<code>None</code>
<code>label</code>	<code>None str</code>	Label for the new image.	<code>None</code>
<code>ndivs</code>	<code>None int</code>	Number of contours and corresponding legend entries.	<code>None</code>
<code>lwidth</code>	<code>None int</code>	Width of the legend.	<code>None</code>
<code>lheight</code>	<code>None int</code>	Height of the legend.	<code>None</code>
<code>decades</code>	<code>None int</code>	Number of decades below the maximum scale for logarithmic mapping.	<code>None</code>
<code>multiplier</code>	<code>None float</code>	Multiplier for the scale (e.g., to convert units).	<code>None</code>
<code>palette</code>	<code>None str</code>	Color palette to use for false color.	<code>None</code>
<code>redv</code>	<code>None str</code>	Expression for mapping values to red.	<code>None</code>
<code>grnv</code>	<code>None str</code>	Expression for mapping values to green.	<code>None</code>
<code>bluv</code>	<code>None str</code>	Expression for mapping values to blue.	<code>None</code>

Returns:

Name	Type	Description
<code>bytes</code>	<code>bytes</code>	Output of falsecolor.

```
genblinds(mat, name, depth, width, height, nslats, angle, rcurv=None)
```

Generate a RADIANCE description of venetian blinds.

Parameters:

Name	Type	Description	Default
<code>mat</code>	<code>str</code>	Material name	<i>required</i>
<code>name</code>	<code>str</code>	Name of the blinds	<i>required</i>
<code>depth</code>	<code>float</code>	Depth of the blinds	<i>required</i>
<code>width</code>	<code>float</code>	Width of the blinds	<i>required</i>
<code>height</code>	<code>float</code>	Height of the blinds	<i>required</i>
<code>nslats</code>	<code>int</code>	Number of slats	<i>required</i>
<code>angle</code>	<code>float</code>	Angle of the slats	<i>required</i>
<code>rcurv</code>	<code>None</code> <code>float</code>	Radius of curvature of the slats, + for upward curvature, - for downward	<code>None</code>

Returns:

Name	Type	Description
<code>bytes</code>	<code>bytes</code>	RADIANCE description of the blinds

Examples:

```
>>> genblinds('mat', 'blinds', 0.1, 0.5, 1.0, 4, 45, 0.1)
```

```
genbox(mat, name, xsiz, ysiz, zsiz, inward=False, beveled=None,
rounded=None, nsegs=None, smoothing=False, waveout=False)
```

Generate a box.

Parameters:

Name	Type	Description	Default
mat	str	material name	<i>required</i>
name	str	box name	<i>required</i>
xsiz	float	size in x dimension	<i>required</i>
ysiz	float	size in y dimension	<i>required</i>
zsiz	float	size in z dimension	<i>required</i>
inward	bool	box facing inward	False
beveled	None float	beveled size	None
rounded	None float	rounded corner size	None
nsegs	None int	number of segments	None
smoothing	bool	to smooth	False
waveout	bool	wavefront (.obj) out	False

Returns:

Type	Description
<code>bytes</code>	the box

```
gendaylit(dt, latitude, longitude, timezone, year=None, dirnorm=None,
diffhor=None, dirhor=None, dirnorm_illum=None, diffhor_illum=None,
solar=False, sky_only=False, silent=False, grefl=None, interval=None)
```

Generates a RADIANCE description of the daylight sources using Perez models for direct and diffuse components.

Parameters:

Name	Type	Description	Default
<code>dt</code>	<code>datetime</code>	python datetime object	<i>required</i>
<code>latitude</code>	<code>float</code>	latitude in degrees	<i>required</i>
<code>longitude</code>	<code>float</code>	longitude in degrees	<i>required</i>
<code>timezone</code>	<code>int</code>	standard meridian timezone, e.g., 120 for PST	<i>required</i>
<code>year</code>	<code>None int</code>	Need to set it explicitly, won't use year in datetime object	<code>None</code>
<code>dirnorm</code>	<code>None float</code>	direct normal irradiance	<code>None</code>
<code>diffhor</code>	<code>None float</code>	diffuse horizontal irradiance	<code>None</code>
<code>dirhor</code>	<code>None float</code>	direct horizontal irradiance, either this or dirnorm	<code>None</code>

Name	Type	Description	Default
<code>dirnorm_illum</code>	<code>None</code> <code>float</code>	direct normal illuminance	<code>None</code>
<code>diffhor_illum</code>	<code>None</code> <code>float</code>	diffuse horizontal illuminance	<code>None</code>
<code>solar</code>	<code>bool</code>	if True, include solar position	<code>False</code>
<code>sky_only</code>	<code>bool</code>	sky description only	<code>False</code>
<code>silent</code>	<code>bool</code>	supress warnings,	<code>False</code>
<code>greffl</code>	<code>None</code> <code>float</code>	ground reflectance	<code>None</code>
<code>interval</code>	<code>None</code> <code>int</code>	interval for epw data	<code>None</code>

Returns:

Type	Description
<code>bytes</code>	output of gendaylit

```
gendaymtx(weather_data, verbose=False, header=True, average=False,
sun_only=False, sky_only=False, sun_file=None, sun_mods=None,
daylight_hours_only=False, dryrun=False, sky_color=None,
ground_color=None, rotate=None, outform=None, onesun=False,
solar_radiance=False, mfactor=1)
```

Generate an annual Perez sky matrix from a weather tape.

Parameters:

Name	Type	Description	Default
weather_data	str Path bytes	weather data	<i>required</i>
mfactor	int	multiplication factor	1
verbose	bool	verbose	False
header	bool	header	True
average	bool	average	False
sun_only	bool	sun only	False
sky_only	bool	sky only	False
sun_file	None str	sun file	None
sun_mods	None str	sun mods	None
daylight_hours_only	bool	daylight hours only	False
dryrun	bool	dryrun	False
sky_color	None list[float]	sky color	None
ground_color	None list[float]	ground color	None
rotate	None float	rotate	None
outform	None str	outform	None
onesun	bool	onesun	False
solar_radiance	bool	solar radiance	False

Returns:

Name	Type	Description
<code>bytes</code>	<code>bytes</code>	output of gendaymtx

```
gensdaymtx(weather_data, verbose=False, header=True, sun_only=False,
sky_only=False, daylight_hours_only=False, ground_reflectance=None,
rotate=None, outform=None, onesun=False, mfactor=1, nthreads=1)
```

Generate an annual spectral sky matrix from a weather tape.

Parameters:

Name	Type	Description	Default
<code>weather_data</code>	<code>str</code> <code>Path</code> <code>bytes</code>	weather data	<i>required</i>
<code>mfactor</code>	<code>int</code>	multiplication factor	<code>1</code>
<code>verbose</code>	<code>bool</code>	verbose	<code>False</code>
<code>header</code>	<code>bool</code>	header	<code>True</code>
<code>sun_only</code>	<code>bool</code>	sun only	<code>False</code>
<code>sky_only</code>	<code>bool</code>	sky only	<code>False</code>
<code>daylight_hours_only</code>	<code>bool</code>	daylight hours only	<code>False</code>
<code>ground_reflectance</code>	<code>None</code> <code>list[float]</code>	ground color	<code>None</code>
<code>rotate</code>	<code>None</code> <code>float</code>	rotate	<code>None</code>
<code>outform</code>	<code>None</code> <code>str</code>	outform	<code>None</code>

Name	Type	Description	Default
<code>onesun</code>	<code>bool</code>	onesun	<code>False</code>
<code>nthreads</code>	<code>int</code>	number of threads to use for precomputations	<code>1</code>

Returns:

Name	Type	Description
<code>bytes</code>	<code>bytes</code>	output of gensdaymtx

```
gensky(dt=None, latitude=None, longitude=None, timezone=None,
altitude=None, azimuth=None, year=None, sunny_with_sun=False,
sunny_without_sun=False, cloudy=False, intermediate_with_sun=False,
intermediate_without_sun=False, uniform=False,
ground_reflectance=None, zenith_brightness=None,
horizontal_brightness=None, solar_radiance=None,
horizontal_direct_irradiance=None, turbidity=None)
```

Generate a RADIANCE description of the sky.

Parameters:

Name	Type	Description	Default
<code>dt</code>	<code>None</code> <code>datetime</code>	datetime object, mutually exclusive with altitude and azimuth	<code>None</code>
<code>latitude</code>	<code>None</code> <code>float</code>	latitude, only apply if dt is not None	<code>None</code>
<code>longitude</code>	<code>None</code> <code>float</code>	longitude, only apply if dt is not None	<code>None</code>

Name	Type	Description	Default
timezone	None int	timezone, only apply if dt is not None	None
altitude	None float	solar altitude, mutally exclusive with dt	None
azimuth	None float	solar azimuth, mutally exclusive with dt	None
year	None int	year, only apply if dt is not None	None
sunny_with_sun	bool	sunny with sun	False
sunny_without_sun	bool	sunny without sun	False
cloudy	bool	CIE overcast sky	False
intermediate_with_sun	bool	intermediate with sun	False
intermediate_without_sun	bool	intermediate without sun	False
uniform	bool	uniform sky	False
ground_reflectance	None float	ground reflectance	None
zenith_brightness	None float	zenith brightness in watts/steradian/meter^2	None
horizontal_brightness	None float	horizontal brightness in watts/metere^2	None
solar_radiance	None float	solar radiance in watts/steradian/meter^2	None

Name	Type	Description	Default
<code>horizontal_direct_irradiance</code>	<code>None</code> <code>float</code>	horizontal direct irradiance in watts/meter^2	<code>None</code>
<code>turbidity</code>	<code>None</code> <code>float</code>	turbidity factor	<code>None</code>

Returns:

Name	Type	Description
<code>str</code>	<code>bytes</code>	output of gensky

```
genssky(dt, latitude=37.7, longitude=122.2, timezone=120, year=None,
res=64, cloud_cover=0.0, ground_reflectance=0.2,
broadband_aerosol_optical_depth=0.115, mie_file=None, nthreads=1,
out_dir='.', out_name='out', dir_norm_illum=None,
diff_hor_illum=None)
```

Generate a RADIANCE description of the spectral sky.

Parameters:

Name	Type	Description	Default
<code>dt</code>	<code>datetime</code>	datetime object, mutually exclusive with altitude and azimuth	<i>required</i>
<code>latitude</code>	<code>float</code>	latitude, only apply if dt is not None	<code>37.7</code>

Name	Type	Description	Default
longitude	float	longitude, only apply if dt is not None	122.2
timezone	int	timezone, only apply if dt is not None	120
year	None int	year, only apply if dt is not None	None
res	int	hsr image resolution, default: 64,	64
cloud_cover	float	cloud cover 0.0: clear, 1.0: overcast,	0.0
ground_reflectance	float	default: 0.2,	0.2
broadband_aerosol_optical_depth	float	default: 0.115,	0.115
mie_file	None str	mie scattering coefficient source file	None
nthreads	int	number of threads used for precomputation, default:1,	1
out_dir	str	directory to save precomputed data that can be reused, default to current working directory. This can be changed to RAYPATH for cross-section data reused.	','

Name	Type	Description	Default
<code>out_name</code>	<code>str</code>	output file name, defaultl: "out"	<code>'out'</code>
<code>dir_norm_illum</code>	<code>None</code> <code>float</code>	direct normal illuminance to calibrate the output against,	<code>None</code>
<code>diff_hor_illum</code>	<code>None</code> <code>float</code>	diffuse horizontal illuminance to calibrate the output against.	<code>None</code>

Returns:

Name	Type	Description
<code>str</code>	<code>bytes</code>	output of gensky

```
get_default_ray_params(*args, **kwargs)
```

`get_default_ray_params()` -> `pyradiance.rad_params.RayParams`

```
get_header(inp, dimension=False)
```

Get header information from a Radiance file.

Parameters:

Name	Type	Description	Default
<code>inp</code>	<code>str</code> <code>Path</code> <code>bytes</code>	input file or bytes	<i>required</i>

Returns:

Name	Type	Description
<code>bytes</code>	<code>bytes</code>	header

```
get_image_dimensions(image)
```

Get the dimensions of an image.

Parameters:

Name	Type	Description	Default
<code>image</code>	<code>str Path bytes</code>	image file path or image bytes	<i>required</i>

Returns:

Type	Description
<code>tuple[int, int]</code>	Tuple[int, int]: width and height

```
get_ray_params_args(*args, **kwargs)
```

`get_ray_params_args(arg: pyradiance.rad_params.RayParams, /) -> list`

Returns a list of strings given a RayParams object

```
get_view_args(*args, **kwargs)
```

`get_view_args(arg: pyradiance.rad_params.View, /) -> list`

Returns a list of string for the view

```
getinfo(*inputs, dimension_only=False, dimension=False,
strip_header=False, replace='', append='', command='')
```

Get header information from a Radiance file.

Parameters:

Name	Type	Description	Default
<code>inputs</code>	<code>tuple[str Path bytes, ...]</code>	input file or bytes	<code>()</code>
<code>dimension_only</code>	<code>bool</code>	return only the dimension	<code>False</code>
<code>dimension</code>	<code>bool</code>	return the dimension	<code>False</code>
<code>strip_header</code>	<code>bool</code>	strip header from the output	<code>False</code>
<code>replace</code>	<code>str</code>	replace the header with this string	<code>''</code>
<code>append</code>	<code>str</code>	append this string to the header	<code>''</code>
<code>command</code>	<code>str</code>	command to use to get the header	<code>''</code>

Returns:

Type	Description
<code>bytes</code>	getinfo output

```
ies2rad(*inp, libdir=None, prefdir=None, outname=None, stdout=False,
units=None, radius=None, instancing_geometry=False, lampdat=None,
lamp_type=None, lamp_color=None, set_default_lamp_color=None,
multiply_factor=None)
```

Convert IES luminaire data to RADIANCE description.

Parameters:

Name	Type	Description	Default
<code>inp</code>	<code>tuple[str Path, ...]</code>	Path to IES file	<code>()</code>
<code>libdir</code>	<code>None str</code>	Path to directory where the library files are located.	<code>None</code>
<code>prefdir</code>	<code>None str</code>	Set the library subdirectory.	<code>None</code>
<code>outname</code>	<code>None str</code>	Output file name root.	<code>None</code>
<code>stdout</code>	<code>bool</code>	Print the output to stdout.	<code>False</code>
<code>units</code>	<code>None str</code>	Set the units of the output file.	<code>None</code>
<code>radius</code>	<code>None float</code>	ignore the crude geometry given by the IES input file and use instead an illum sphere with radius rad.	<code>None</code>
<code>instancing_geometry</code>	<code>bool</code>	compile MGF detail geometry into a separate octree and create a single instance referencing it.	<code>False</code>
<code>lampdat</code>	<code>None str</code>	map lamp name to xy chromaticity and lumen depreciation data.	<code>None</code>
<code>lamp_type</code>	<code>None str</code>	lamp type.	<code>None</code>
<code>lamp_color</code>	<code>None str</code>	set lamp color to red green blue if lamp type is unknown.	<code>None</code>

Name	Type	Description	Default
<code>set_default_lamp_color</code>	<code>None str</code>	set default lamp color according to the entry for lamp in the lookup table.	<code>None</code>
<code>multiply_factor</code>	<code>None float</code>	multiply all output quantities by this factor. This is the best way to scale fixture brightness for different lamps.	<code>None</code>

Returns:

Type	Description
<code>bytes</code>	The output of the command

```
load_material_smd(file, roughness=0.0, spectral=False, metal=False)
```

Generate Radiance primitives from csv file from spectral material database (spectraldb.com).

Parameters:

Name	Type	Description	Default
<code>file</code>	<code>Path</code>	Path to .csv file	<i>required</i>
<code>roughness</code>	<code>float</code>	Roughness of material	<code>0.0</code>
<code>spectral</code>	<code>bool</code>	Output spectral primitives	<code>False</code>
<code>metal</code>	<code>bool</code>	Whether material is metal	<code>False</code>

Returns:

Type	Description
<code>list[Primitive]</code>	A list of primitives

```
mgf2rad(*inp, matfile=None, mult=None, dist=None)
```

Convert Materials and Geometry Format file to RADIANCE description.

Parameters:

Name	Type	Description	Default
<code>inp</code>	<code>str</code>	Path to MGF file	<code>()</code>
<code>matfile</code>	<code>None str</code>	Path to material file where the translated materials will be written.	<code>None</code>
<code>mult</code>	<code>None float</code>	multiplier for all the emission values	<code>None</code>
<code>dist</code>	<code>None float</code>	glow distance (in meters) for all emitting surfaces.	<code>None</code>

Returns:

Type	Description
<code>bytes</code>	The output of the command

```
mkillum(inp, octree, nproc=1, params=None)
```

Compute illum sources for a RADIANCE scene

Parameters:

Name	Type	Description	Default
<code>inp</code>	<code>bytes</code>	input file content as bytes	<i>required</i>
<code>octree</code>	<code>str</code> <code>Path</code>	octree file	<i>required</i>
<code>nproc</code>	<code>int</code>	number of processes	<code>1</code>
<code>params</code>	<code>None</code> <code>Sequence[str]</code>	additional parameters	<code>None</code>

Returns:

Type	Description
<code>bytes</code>	Output of mkillum in bytes

```
mkpmap(octree, global_map=None, caustic_map=None, volume_map=None,
direct_map=None, contrib_map=None, pre_global_map=None,
predistrib=None, rect_region=None, sphere_region=None,
maxbounce=None, maxprepass=None, port_mod=None, port_modfile=None,
precomp=None, seed=None, virtual_mod=None, virtual_modfile=None,
amb_excl_mod=None, amb_excl_modfile=None, amb_incl_mod=None,
amb_incl_modfile=None, backface_vis=False, sample_res=None,
partition_size=None, progress_file=None, overwrite=False,
maxdist=None, scattering_albedo=None, extinction_coefficient=None,
scattering_eccentricity=None, nproc=1, progress_interval=None)
```

Mkpmap takes a RADIANCE scene description as an octree and performs Monte Carlo forward path tracing from the light sources, depositing indirect ray hitpoints along with their energy (flux) as "photons". The resulting localised energy distribution represents a global illumination solution which is written to a file for subsequent evaluation by `rpict(1)`, `rtrace(1)` and `rvu(1)` in a backward raytracing pass. The photon map(s) can be reused for multiple viewpoints and sensor locations as long as the geometry remains unchanged. Args: `octree`: Octree file path. `global_map`: Global map file path and number of photons. `caustic_map`: Caustic map file path and number of photons. `volume_map`: Volume map file path and number of photons. `direct_map`: Direct map file path and

number of photons. contrib_map: Contribution map file path and number of photons.
pre_global_map: Precomputed global map file path, number of photons and bandwidth. predistrib: Photon predistribution factor. rect_region: Rectangular region sphere_region: Spherical region
maxbounce: Maximum number of bounces. maxprepass: Maximum number of iterations of distributoin prepass before terminating. port_mod: Specifies a modifier to act as a photon port.
port_modfile: File with modifiers to act as photon ports. precomp: Fraction of global photons to precompute. seed: Random seed. virtual_mod: Specifies a modifier to act as a virtual source.
virtual_modfile: File with modifiers to act as virtual sources. amb_excl_mod: Specifies a modifier to exclude from ambient calculation. amb_excl_modfile: File with modifiers to exclude from ambient calculation. amb_incl_mod: Specifies a modifier to include in ambient calculation.
amb_incl_modfile: File with modifiers to include in ambient calculation. backface_vis: Backface visibility. sample_res: Sample resolution.

```
obj2mesh(inp, matfiles=None, matlib=None, objlim=9, maxres=16384,
silent=False, stats=False)
```

Convert Wavefront .OBJ file to RADIANCE description.

Parameters:

Name	Type	Description	Default
inp	bytes str Path	Path to OBJ file, or bytes	<i>required</i>
matfiles	None Sequence[str]	Radiance material files to use	None
matlib	None str	Radiance material library file, RAYPATH are searched	None
objlim	int	the maximum surface setsize for each voxel	9
maxres	int	Maximum octree resolution	16384
silent	bool	Suppress warnings	False

Name	Type	Description	Default
<code>stats</code>	<code>bool</code>	Print statistics	<code>False</code>

Returns:

Type	Description
<code>bytes</code>	The converted RADIANCE scene description in bytes

```
obj2rad(inp, quallist=False, flatten=False, mapfile=None,
objname=None)
```

Convert Wavefront .OBJ file to RADIANCE description.

Parameters:

Name	Type	Description	Default
<code>inp</code>	<code>bytes str</code> <code> Path</code>	Path to OBJ file	<i>required</i>
<code>quallist</code>	<code>bool</code>	Produce a list of qualifiers from which to construct a mapping for the given .OBJ file.	<code>False</code>
<code>flatten</code>	<code>bool</code>	Flatten all faces, effectively ignoring vertex normal information.	<code>False</code>
<code>mapfile</code>	<code>None str</code>	Mapping rules files for assigning material names for the surfaces.	<code>None</code>
<code>objname</code>	<code>None str</code>	Specify the name of this object, though it will be overridden by any "o" statements in the input file. If this option is absent, and there are no "o" statements, obj2rad will	<code>None</code>

Name	Type	Description	Default
		attempt to name surfaces based on their group associations.	

Returns:

Type	Description
<code>bytes</code>	The converted RADIANCE scene description in bytes

```
oconv(*paths, warning=True, stdin=None, frozen=False, octree=None)
```

Run Radiance oconv tool to build an octree.

Parameters:

Name	Type	Description	Default
<code>paths</code>	<code>str</code>	list of Radiance files	<code>()</code>
<code>warning</code>	<code>bool</code>	if False, warnings will be suppressed	<code>True</code>
<code>stdin</code>	<code>None</code> <code>bytes</code>	if not None, stdin will be used	<code>None</code>
<code>frozen</code>	<code>bool</code>	if True, the octree will be frozen	<code>False</code>
<code>octree</code>	<code>None</code> <code>str</code> <code>Path</code>	if provided, the resulting octree incorporate existing one	<code>None</code>

Returns:

Name	Type	Description
<code>bytes</code>	<code>bytes</code>	output of oconv

```
pabopto2bsdf(*inp, nproc=1, symmetry=None, angle=None, reverse=False)
```

Convert BSDF measurements to a scattering interpolant representation.

Parameters:

Name	Type	Description	Default
<code>inp</code>	<code>str</code>	pab-opto Mountain files, need two or more.	<code>()</code>
<code>nproc</code>	<code>int</code>	number of processors to use.	<code>1</code>
<code>symmetry</code>	<code>None</code> <code>str</code>	BSDF symmetry, which is one of "isotropic", "quadrilateral", "bilateral", "up", or "anisotropic". Any of these may be abbreviated with as little as a single letter, and case is ignored.	<code>None</code>
<code>angle</code>	<code>None</code> <code>float</code>	cull scattered measurements that are nearer to grazing than the given angle in degrees. If the word "auto" (which can be abbreviated as 'a' or 'A') is given instead of an angle, then the near-grazing angle will be determined by the lowest incident angle measurement present in the input data. This is sometimes necessary to eliminate noise and edge effects that some measurements exhibit near grazing.	<code>None</code>
<code>reverse</code>	<code>bool</code>	reverses the assumed sample orientation front-to-back, and is discussed below under the "#intheta" header entry.	<code>False</code>

Returns:

Type	Description
<code>bytes</code>	SIR data in bytes

```
parse_primitive(pstr)
```

Parse Radiance primitives inside a file path into a list of dictionary.

Parameters:

Name	Type	Description	Default
<code>pstr</code>	<code>str</code>	A string of Radiance primitives.	<i>required</i>

Returns:

Type	Description
<code>list[Primitive]</code>	list of primitives

```
parse_view(*args, **kwargs)
```

`parse_view(arg: str, /) -> pyradiance.rad_params.View`

Parse a view string into a View object

```
pcompos(inputs, pos=None, xres=None, yres=None, spacing=0,
background=None, anchors=None, header=True, lower_threshold=None,
upper_threshold=None, label=None, label_height=None, ncols=None,
anchor_point=None)
```

Composite Radiance pictures

Parameters:

Name	Type	Description	Default
<code>inputs</code>	<code>Sequence[Path str bytes]</code>	list of input files	<i>required</i>

Name	Type	Description	Default
<code>pos</code>	<code>None Sequence[Sequence[float]]</code>	list of positions	<code>None</code>
<code>xres</code>	<code>None int</code>	horizontal resolution	<code>None</code>
<code>yres</code>	<code>None int</code>	vertical resolution	<code>None</code>
<code>spacing</code>	<code>int</code>	spacing between images	<code>0</code>
<code>background</code>	<code>None tuple[float, float, float]</code>	background color	<code>None</code>
<code>anchors</code>	<code>None Sequence[str]</code>	list of anchors	<code>None</code>
<code>header</code>	<code>bool</code>	set to False if want to reduce header	<code>True</code>
<code>lower_threashold</code>	<code>None float</code>	lower threshold	<code>None</code>
<code>upper_threshold</code>	<code>None float</code>	upper threshold	<code>None</code>
<code>label</code>	<code>None str</code>	label	<code>None</code>
<code>label_height</code>	<code>None int</code>	label height	<code>None</code>
<code>ncols</code>	<code>None int</code>	number of columns	<code>None</code>
<code>anchor_point</code>	<code>None Sequence[str]</code>	anchor point	<code>None</code>

Returns:

Name	Type	Description
<code>bytes</code>	<code>bytes</code>	output of pcompos

```
pcond(hdr, human=False, acuity=False, veiling=False, sense=False,
closs=False, center_weighted=False, linear=False, fixfrac=0.0,
fixpoints=None, histo='', expval='', ldmax=100.0, lddyn=100.0,
primaries=None, macbeth='', mapfile='')
```

Condition a Radiance picture.

Parameters:

Name	Type	Description	Default
<code>hdr</code>	<code>Path</code>	input hdr	<i>required</i>
<code>human</code>	<code>bool</code>	Mimic human visual response. This is the same as turning on all acuity, veiling, sensitivity, and color loss.	<code>False</code>
<code>acuity</code>	<code>bool</code>	Defocus darker region.	<code>False</code>
<code>veiling</code>	<code>bool</code>	Add veiling glare.	<code>False</code>
<code>sense</code>	<code>bool</code>	Use human contrast sensitivity, simulating eye internal scattering.	<code>False</code>
<code>center_weighted</code>	<code>bool</code>	Use center-weighted average for acuity and veiling.	<code>False</code>
<code>linear</code>	<code>bool</code>	Use a linear reponse function instead of the standard dynamic range compression. This preseves the extremas.	<code>False</code>

Name	Type	Description	Default
<code>fixfrac</code>	<code>float</code>	Fixation fraction for acuity and veiling.	<code>0.0</code>
<code>histo</code>	<code>str</code>	Histogram file for acuity and veiling.	<code>''</code>
<code>expval</code>	<code>str</code>	Exposure value for acuity and veiling.	<code>''</code>
<code>ldmax</code>	<code>float</code>	Maximum luminance for acuity and veiling.	<code>100.0</code>
<code>lddyn</code>	<code>float</code>	Luminance dynamic range for acuity and veiling.	<code>100.0</code>
<code>primaries</code>	<code>None</code> <code>list[float]</code>	Color primaries for color loss.	<code>None</code>
<code>macbeth</code>	<code>str</code>	Macbeth chart file for color loss.	<code>''</code>
<code>mapfile</code>	<code>str</code>	Color map file for color loss.	<code>''</code>

Returns:

Name	Type	Description
<code>bytes</code>	<code>bytes</code>	output of pcond

```
pfilt(image, xres=None, yres=None, pixel_aspect=0, pa_correct=False,
exposure=0, lamp=None, lampdat=None, one_pass=False,
gaussian_filter_radius=None, limitfrac=None, hot_threshold=100,
star_points=0, star_spread=0.0001, average_hot=False)
```

filter a Radiance picture. By default, it uses two passes on the input, using a box filter.

Parameters:

Name	Type	Description	Default
image	str Path bytes	input image	<i>required</i>
xres	None str	horizontal resolution	None
yres	None str	vertical resolution	None
pixel_aspect	float	pixel aspect ratio	0
exposure	None float	exposure value	0
lamp	None str	lamp file	None
lampdat	None str	lamp data file	None
one_pass	bool	use one pass filter	False
gaussian_filter_radius	None float	gaussian filter radius	None
limitfrac	None float	limit fraction	None
hot_threshold	float	Set intensity considered 'hot', default 100 watts/sr/m2	100
star_points	int	Number of points on a start pattern.	0
star_spread	float	star pattern spread	0.0001
average_hot	bool	average hot spots	False

Returns:

Name	Type	Description
<code>bytes</code>	<code>bytes</code>	output of pfilt

```
pkgbsdf(*xml, frozen=False, stdout=False)
```

Pacakge BSDFs provided as XML for Radiance.

Parameters:

Name	Type	Description	Default
<code>xml</code>	<code>tuple[str Path, ...]</code>	Path to XML files	<code>()</code>
<code>frozen</code>	<code>bool</code>	produce frozen octree instance for any detail geometry.	<code>False</code>
<code>stdout</code>	<code>bool</code>	print the output to stdout, only works for a single XML input.	<code>False</code>

Returns:

Type	Description
<code>None bytes</code>	The output of the command

```
pvalue(pic, unique=False, original=False, header=True, resstr=True,
skip=0, exposure=0, gamma=1.0, dataonly=False, outform='',
reverse_rgb=False, interleaving=True, brightness=False,
outprimary=None)
```

convert RADIANCE picture to/from alternate formats Pvalue converts the pixels of a RADIANCE picture to or from another format. In the default mode, pixels are sent to the standard output, one per line, in the following ascii format: xpos ypos red green blue

Parameters:

Name	Type	Description	Default
<code>pic</code>	<code>Path str bytes</code>	hdr file path. Either path or stdin is used, path takes precedence.	<i>required</i>
<code>unique</code>	<code>bool</code>	if True, only unique values will be returned	<code>False</code>
<code>original</code>	<code>bool</code>	if True, print original values, before exposure compensation	<code>False</code>
<code>header</code>	<code>bool</code>	if True, header will be returned	<code>True</code>
<code>resstr</code>	<code>bool</code>	if True, resolution string will be returned	<code>True</code>
<code>skip</code>	<code>int</code>	number of bytes to skip	<code>0</code>
<code>exposure</code>	<code>int</code>	exposure value	<code>0</code>
<code>gamma</code>	<code>float</code>	gamma value	<code>1.0</code>
<code>dataonly</code>	<code>bool</code>	if True, only data will be returned	<code>False</code>
<code>outform</code>	<code>str</code>	output data format	<code>''</code>
<code>reverse_rgb</code>	<code>bool</code>	if True, RGB values will be reversed	<code>False</code>
<code>interleaving</code>	<code>bool</code>	if True, interleaving will be used	<code>True</code>
<code>brightness</code>	<code>bool</code>	if True, only brightness will be returned	<code>False</code>
<code>outprimary</code>	<code>None str</code>	output color primaries	<code>None</code>

Returns:

Name	Type	Description
<code>bytes</code>	<code>bytes</code>	output of pvalue

```
pvaluer(pic, xres=None, yres=None, inform='a', resstr=True,
dataonly=False, header=True, primaries=None, pxyz=False)
```

Run Radiance pvalue tools reverse mode: constructing a image from pixel values.

Parameters:

Name	Type	Description	Default
<code>pic</code>	<code>Path str bytes</code>	Path or bytes of the input pixel data.	<i>required</i>
<code>xres</code>	<code>None int</code>	X resolution.	<code>None</code>
<code>yres</code>	<code>None int</code>	Y resolution.	<code>None</code>
<code>inform</code>	<code>str</code>	input data format. Default is "a" for ascii.	<code>'a'</code>
<code>header</code>	<code>bool</code>	Set to True if the picture file has a header. Default is False.	<code>True</code>
<code>primaries</code>	<code>None list[float]</code>	list of primaries for XYZ calculation. Default is None.	<code>None</code>
<code>pxyz</code>	<code>bool</code>	Set to True to calculate XYZ values. Default is False.	<code>False</code>

Returns:

Type	Description
<code>bytes</code>	Bytes of the pvalue output

```
ra_ppm(inp, gamma=2.2, greyscale=False, reverse=False, exposure=0,
ascii=False, outscale=255)
```

convert RADIANCE picture to/from a Poskanzer Portable Pixmap

Parameters:

Name	Type	Description	Default
<code>inp</code>	<code>str</code> <code>Path</code> <code>bytes</code>	Path or bytes to input picture file.	<i>required</i>
<code>gamma</code>	<code>float</code>	Gamma value for the output image. Default is 2.2.	<code>2.2</code>
<code>reverse</code>	<code>bool</code>	Set to True to invoke a reverse conversion, from a PPM to a RADIANCE picture. Default is False.	<code>False</code>
<code>exposure</code>	<code>int</code>	Exposure value for the output image. Default is 0.	<code>0</code>
<code>ascii</code>	<code>bool</code>	Set to True to use ASCII Pixmap output. Default is False.	<code>False</code>
<code>outscale</code>	<code>int</code>	Output scale value. Default is 255.	<code>255</code>

Returns:

Name	Type	Description
<code>bytes</code>	<code>bytes</code>	output of <code>ra_ppm</code>

```
ra_tiff(inp, out=None, gamma=2.2, greyscale=False, lzw=False,
sgilog=False, sgilog24=False, iieee32=False, primary=False,
exposure=0, reverse=False, xyze=False)
```

ra_tiff - convert RADIANCE picture to/from a TIFF color or greyscale image

Parameters:

Name	Type	Description	Default
inp	str Path bytes	Path or bytes to input picture file.	<i>required</i>
out	None str	Path to output file, required when output is a TIFF file.	None
gamma	float	Gamma value for the output image. Default is 2.2.	2.2
greyscale	bool	Set to True to convert to greyscale. Default is False.	False
lzw	bool	Set to True to use LZW compression. Default is False.	False
sgilog	bool	Set to True to use SGI log compression. Default is False.	False
sgilog24	bool	Set to True to use SGI log 24 compression. Default is False.	False
ieeee32	bool	Set to True to use IEEE 32-bit floating point compression.	False
primary	bool	Set to True to use 16-bit/primary output. Default is False.	False
reverse	bool	Set to True to invoke a reverse conversion, from a TIFF to a RADIANCE	False

Name	Type	Description	Default
		picture. Default is False.	
<code>xyze</code>	<code>bool</code>	Set to True to use XYZE output when invoking a reverse conversion. Default is False.	<code>False</code>

Returns:

Name	Type	Description
<code>bytes</code>	<code>None</code> <code>bytes</code>	output of <code>ra_tiff</code>

```
rcalc(inp, sep=None, inform=None, incount=1, outform=None,
passive=False, pass_negative=False, inlimit=None, outlimit=None,
source=None, expr=None, assign=None, exact_only=False,
ignore_newlines=False, silent=False)
```

Run `rcalc`

Parameters:

Name	Type	Description	Default
<code>inp</code>	<code>str</code> <code>Path</code> <code>bytes</code>	input data	<i>required</i>
<code>sep</code>	<code>None</code> <code>str</code>	separator	<code>None</code>
<code>inform</code>	<code>None</code> <code>str</code>	input format	<code>None</code>
<code>incount</code>	<code>int</code>	input count	<code>1</code>
<code>outform</code>	<code>None</code> <code>str</code>	output format	<code>None</code>
<code>passive</code>	<code>bool</code>	passive mode	<code>False</code>

Name	Type	Description	Default
<code>pass_negative</code>	<code>bool</code>	pass negative	<code>False</code>
<code>inlimit</code>	<code>None int</code>	input limit	<code>None</code>
<code>outlimit</code>	<code>None int</code>	output limit	<code>None</code>
<code>source</code>	<code>None str</code>	source file	<code>None</code>
<code>expr</code>	<code>None str</code>	expression	<code>None</code>
<code>assign</code>	<code>None str</code>	assign	<code>None</code>
<code>exact_only</code>	<code>bool</code>	exact only	<code>False</code>
<code>ignore_newlines</code>	<code>bool</code>	ignore newlines	<code>False</code>
<code>silent</code>	<code>bool</code>	silent	<code>False</code>

Returns:

Name	Type	Description
<code>bytes</code>	<code>bytes</code>	output of rcalc

```
rcode_depth(inp, ref_depth='1.0', inheader=True, outheader=True,
inresolution=True, outresolution=True, xres=None, yres=None,
inform='a', outform='a', decode=False, compute_intersection=False,
per_point=False, depth_file=None, flush=False)
```

Encode/decode 16-bit depth map.

Parameters:

Name	Type	Description	Default
<code>inp</code>	<code>str</code> <code>Path</code> <code>bytes</code>	input file or bytes	<i>required</i>
<code>ref_depth</code>	<code>str</code>	reference distance, can be follow by /unit.	<code>'1.0'</code>
<code>inheader</code>	<code>bool</code>	Set to False to not expect header on input	<code>True</code>
<code>outheader</code>	<code>bool</code>	Set to False to not include header on output	<code>True</code>
<code>inresolution</code>	<code>bool</code>	Set to False to not expect resolution on input	<code>True</code>
<code>outresolution</code>	<code>bool</code>	Set to False to not include resolution on output	<code>True</code>
<code>xres</code>	<code>None</code> <code>int</code>	x resolution	<code>None</code>
<code>yres</code>	<code>None</code> <code>int</code>	y resolution	<code>None</code>
<code>inform</code>	<code>str</code>	input format	<code>'a'</code>
<code>outform</code>	<code>str</code>	output format when decoding	<code>'a'</code>
<code>decode</code>	<code>bool</code>	Set to True to decode instead	<code>False</code>
<code>compute_intersection</code>	<code>bool</code>	Set to True to compute intersection instead	<code>False</code>
<code>per_point</code>	<code>bool</code>	Set to True to compute per point instead of per pixel	<code>False</code>
<code>depth_file</code>	<code>None</code> <code>str</code>	depth file	<code>None</code>

Name	Type	Description	Default
<code>flush</code>	<code>bool</code>	Set to True to flush output	<code>False</code>

Returns:

Name	Type	Description
<code>bytes</code>	<code>bytes</code>	output of <code>rcode_depth</code>

```
rcode_ident(inp, index_size=16, sep='\n', decode=False, header=True,
xres=None, yres=None, resstr=True, identifiers=False, indexes=False,
per_point=False, flush=False)
```

Store identifiers in an indexed map and retrieve from same

Parameters:

Name	Type	Description	Default
<code>inp</code>	<code>str</code> <code>Path</code> <code>bytes</code>	input file or bytes	<i>required</i>
<code>index_size</code>	<code>int</code>	index size	<code>16</code>
<code>sep</code>	<code>str</code>	separator	<code>'\n'</code>
<code>decode</code>	<code>bool</code>	Set to True to decode instead	<code>False</code>
<code>header</code>	<code>bool</code>	Set to False to not to expect header on input; or not to include header on output when decoding	<code>True</code>
<code>xres</code>	<code>None</code> <code>int</code>	x resolution	<code>None</code>
<code>yres</code>	<code>None</code> <code>int</code>	y resolution	<code>None</code>

Name	Type	Description	Default
<code>resstr</code>	<code>bool</code>	Set to False to not include resolution string on output	<code>True</code>
<code>identifiers</code>	<code>bool</code>	Set to True to include identifiers on output	<code>False</code>
<code>indexes</code>	<code>bool</code>	Set to True to instead list identifiers indexes on output	<code>False</code>
<code>per_point</code>	<code>bool</code>	Set to True to compute per point instead of per pixel	<code>False</code>
<code>flush</code>	<code>bool</code>	Set to True to flush output after each identifier	<code>False</code>

Returns:

Name	Type	Description
<code>bytes</code>	<code>bytes</code>	output of <code>rcode_ident</code>

```
rcode_norm(inp, inheader=True, outheader=True, inresolution=True,
outresolution=True, xres=None, yres=None, inform='a', outform='a',
decode=False, per_point=False, norm_file=None, flush=False)
```

Encode/decode 32-bit surface normal map.

Parameters:

Name	Type	Description	Default
<code>inp</code>	<code>str</code> <code>Path</code> <code>bytes</code>	input file or bytes	<i>required</i>

Name	Type	Description	Default
<code>inheader</code>	<code>bool</code>	Set to False to not expect header on input	<code>True</code>
<code>outheader</code>	<code>bool</code>	Set to False to not include header on output	<code>True</code>
<code>inresolution</code>	<code>bool</code>	Set to False to not expect resolution on input	<code>True</code>
<code>outresolution</code>	<code>bool</code>	Set to False to not include resolution on output	<code>True</code>
<code>xres</code>	<code>None int</code>	x resolution	<code>None</code>
<code>yres</code>	<code>None int</code>	y resolution	<code>None</code>
<code>inform</code>	<code>str</code>	input format	<code>'a'</code>
<code>outform</code>	<code>str</code>	output format when decoding	<code>'a'</code>
<code>decode</code>	<code>bool</code>	Set to True to decode instead	<code>False</code>
<code>per_point</code>	<code>bool</code>	Set to True to compute per point instead of per pixel	<code>False</code>
<code>flush</code>	<code>bool</code>	Set to True to flush output	<code>False</code>

Returns:

Name	Type	Description
<code>bytes</code>	<code>bytes</code>	output of <code>rdecode_norm</code>

```
render(scene, view=None, quality='Medium', variability='Medium',
detail='Medium', nproc=1, ncssamp=3, resolution=None, ambbounce=0,
ambcache=True, params=None)
```

Render a scene.

Parameters:

Name	Type	Description	Default
<code>scene</code>	Scene	Scene object.	<i>required</i>
<code>quality</code>	<code>str</code>	Quality level.	<code>'Medium'</code>
<code>variability</code>	<code>str</code>	Variability level.	<code>'Medium'</code>
<code>detail</code>	<code>str</code>	Detail level.	<code>'Medium'</code>
<code>nproc</code>	<code>int</code>	Number of processes to use.	<code>1</code>
<code>ncssamp</code>	<code>int</code>	Number of channels to sample	<code>3</code>
<code>ambbounce</code>	<code>int</code>	Number of ambient bounces.	<code>0</code>
<code>ambcache</code>	<code>bool</code>	Use ambient cache.	<code>True</code>
<code>params</code>	<code>None</code> RayParams	Sampling parameters.	<code>None</code>

Returns:

Type	Description
<code>bytes</code>	<code>tuple[bytes, int, int]</code> : output of render, width, height

```
rfluxmtx(receiver, surface=None, rays=None, params=None, octree=None,
scene=None)
```

Run rfluxmtx command.

Parameters:

Name	Type	Description	Default
receiver	str Path	receiver file path	<i>required</i>
surface	None str Path	input surface file path, mutually exclusive with rays	None
rays	None bytes	input rays bytes, mutually exclusive with surface	None
params	None Sequence[str]	ray tracing parameters	None
octree	None Path str	octree file path	None
scene	None Sequence[Path str]	list of scene files	None

Returns:

Type	Description
bytes	The results of rfluxmtx in bytes

`rlam(*inputs)`

Laminate records from multiple files.

Parameters:

Name	Type	Description	Default
inputs	str Path bytes	list of input files or bytes. There can only be one bytes input.	()

Returns:

Name	Type	Description
<code>bytes</code>	<code>bytes</code>	output of rlam

```
rmtxop(inp, outform='a', transpose=False, scale=None, transform=None,
reflectance=None)
```

Run rmtxop command.

Parameters:

Name	Type	Description	Default
<code>inp</code>	<code>str</code> <code>Path</code> <code>bytes</code>	input	<i>required</i>
<code>outform</code>	<code>str</code>	output format: 'a', 'f', 'd', 'c'	<code>'a'</code>
<code>transpose</code>	<code>bool</code>	whether to transpose matrix	<code>False</code>
<code>scale</code>	<code>None</code> <code>float</code>	scaling factor	<code>None</code>
<code>transform</code>	<code>None</code> <code>Sequence[float]</code>	transform factors for each channel	<code>None</code>

Returns:

Type	Description
<code>bytes</code>	The results of rmtxop in bytes

```
robjutil(inp, radout=False, verbose=False,
remove_texture_coordinates=False, remove_surface_normal=False,
remove_surface_by_modifier=None, keep_surface_by_modifier=None,
```

```
remove_surface_by_group=None, keep_surface_by_group=None,  
epsilon=None, triangulate=False, transform=None)
```

Operate on Wavefront .OBJ file

Parameters:

Name	Type	Description	Default
<code>inp</code>	<code>str</code> <code>Path</code>	Input .OBJ file path	<i>required</i>
<code>radout</code>	<code>bool</code>	Output RADIANCE scene description	<code>False</code>
<code>verbose</code>	<code>bool</code>	Set to True to turn on verbosity	<code>False</code>
<code>remove_texture_coordinates</code>	<code>bool</code>	Remove texture coordinates from the output	<code>False</code>
<code>remove_surface_normal</code>	<code>bool</code>	Remove surface normal from the output	<code>False</code>
<code>remove_surface_by_modifier</code>	<code>None</code> <code>Sequence[str]</code>	Remove surfaces by modifier, mutually exclusive with <code>keep_surface_by_modifier</code>	<code>None</code>
<code>keep_surface_by_modifier</code>	<code>None</code> <code>Sequence[str]</code>	Keep surfaces by modifier, mutually exclusive with <code>remove_surface_by_modifier</code>	<code>None</code>
<code>remove_surface_by_group</code>	<code>None</code> <code>Sequence[str]</code>	Remove surfaces by group/object, mutually exclusive with <code>keep_surface_by_group</code>	<code>None</code>
<code>keep_surface_by_group</code>	<code>None</code> <code>Sequence[str]</code>	Keep surfaces by group/object, mutually	<code>None</code>

Name	Type	Description	Default
		exclusive with remove_surface_by_group	
<code>epsilon</code>	<code>None</code> <code>float</code>	Coalesce vertices that are within the given epsilon	<code>None</code>
<code>triangulate</code>	<code>bool</code>	Turns all faces with 4 or more sides into triangles	<code>False</code>
<code>transform</code>	<code>None</code> <code>str</code>	Transform the input, using xform CLI syntax.	<code>None</code>

Returns:

Type	Description
<code>bytes</code>	The output of the command

```
rpict(view, octree, xres=None, yres=None, report=0, report_file=None,
params=None)
```

Get rpict command.

Parameters:

Name	Type	Description	Default
<code>view</code>	<code>Sequence[str]</code>	A list of view parameters in strings.	<i>required</i>
<code>octree</code>	<code>Path</code> <code>str</code>	A path to octree file.	<i>required</i>
<code>xres</code>	<code>None</code> <code>int</code>	X resolution.	<code>None</code>

Name	Type	Description	Default
<code>yses</code>	<code>None int</code>	Y resolution.	<code>None</code>
<code>report</code>	<code>float</code>	Report.	<code>0</code>
<code>report_file</code>	<code>None Path</code>	Report file.	<code>None</code>
<code>params</code>	<code>None Sequence[str]</code>	A list of additional parameters.	<code>None</code>

Returns:

Type	Description
<code>bytes</code>	A bytes object.

```
rsensor(sensor, sensor_view=None, direct_ray=None, ray_count=None,
octree=None, nproc=1, params=None)
```

Compute sensor signal from a RADIANCE scene

Parameters:

Name	Type	Description	Default
<code>sensor</code>	<code>Sequence[str Path]</code>	Sensor file	<i>required</i>
<code>sensor_view</code>	<code>None Sequence[str Path]</code>	Sensor view file	<code>None</code>
<code>direct_ray</code>	<code>None Sequence[int]</code>	The number of rays sent to each light source per sensor	<code>None</code>

Name	Type	Description	Default
<code>ray_count</code>	<code>None</code> <code>Sequence[int]</code>	The number of ray samples sent at random	<code>None</code>
<code>octree</code>	<code>None</code> <code>str</code> <code>Path</code>	Octree file	<code>None</code>
<code>nproc</code>	<code>int</code>	Number of processors to use	<code>1</code>
<code>params</code>	<code>None</code> <code>Sequence[str]</code>	Additional parameters for rsensor command	<code>None</code>

Returns:

Type	Description
<code>bytes</code>	Output of rsensor command

```
rtrace(rays, octree, header=True, inform='a', outform='a',
irradiance=False, irradiance_lambertian=False, outspec=None,
trace_exclude='', trace_include='', trace_exclude_file=None,
trace_include_file=None, uncorrelated=False, xres=None, yres=None,
nproc=None, params=None, report=False, version=False)
```

Run rtrace.

Parameters:

Name	Type	Description	Default
<code>rays</code>	<code>bytes</code>	A string of bytes representing the input rays.	<i>required</i>
<code>octree</code>	<code>Path</code> <code>str</code>	Path to octree file.	<i>required</i>

Name	Type	Description	Default
<code>header</code>	<code>bool</code>	A boolean to indicate if the header should be included in the output.	<code>True</code>
<code>inform</code>	<code>str</code>	Input format. Default is 'a'.	<code>'a'</code>
<code>outform</code>	<code>str</code>	Output format. Default is 'a'.	<code>'a'</code>
<code>irradiance</code>	<code>bool</code>	A boolean to indicate if irradiance should be calculated.	<code>False</code>
<code>irradiance_lambertian</code>	<code>bool</code>	A boolean to indicate if irradiance should be calculated using Lambertian assumption.	<code>False</code>
<code>outspec</code>	<code>None str</code>	Output specification. Default is None.	<code>None</code>
<code>trace_exclude</code>	<code>str</code>	A string of space separated material names to exclude from the trace.	<code>''</code>
<code>trace_include</code>	<code>str</code>	A string of space separated material names to include in the trace.	<code>''</code>
<code>trace_exclude_file</code>	<code>None str Path</code>	Path to a file containing material names to exclude from the trace.	<code>None</code>
<code>trace_include_file</code>	<code>None str Path</code>	Path to a file containing material names to include in the trace.	<code>None</code>

Name	Type	Description	Default
<code>uncorrelated</code>	<code>bool</code>	A boolean to indicate if uncorrelated sampling should be used.	<code>False</code>
<code>xres</code>	<code>None int</code>	X resolution of the output image.	<code>None</code>
<code>yres</code>	<code>None int</code>	Y resolution of the output image.	<code>None</code>
<code>nproc</code>	<code>None int</code>	Number of processors to use.	<code>None</code>

Returns:

Type	Description
<code>bytes</code>	A string of bytes representing the output of rtrace.

`spec_xyz(*args, **kwargs)`

`spec_xyz(arg0: collections.abc.Sequence[float], arg1: float, arg2: float, /) -> tuple`

`total(inp, mean=False, sumpower=0, multiply=False, find_max=False, find_min=False, inform=None, incount=1, outform=None, substep=None, substep_reset=True, inlimit=None, outlimit=None, sep=None)`

Sum up columns.

Parameters:

Name	Type	Description	Default
<code>inp</code>	<code>None str Path</code>	input file or bytes	<i>required</i>

Name	Type	Description	Default
<code>mean</code>	<code>bool</code>	if True, the mean value will be calculated	<code>False</code>
<code>sumpower</code>	<code>int</code>	the power of the sum, mutually exclusive with multiply, find_max, and find_min	<code>0</code>
<code>multiply</code>	<code>bool</code>	if True, the values will be multiplied, mutually exclusive with sumpower, find_max, and find_min	<code>False</code>
<code>find_max</code>	<code>bool</code>	if True, the maximum value will be found, mutually exclusive with sumpower, multiply, and find_min	<code>False</code>
<code>find_min</code>	<code>bool</code>	if True, the minimum value will be found, mutually exclusive with sumpower, multiply, and find_max	<code>False</code>
<code>inform</code>	<code>None str</code>	input format	<code>None</code>
<code>incount</code>	<code>int</code>	number of input values	<code>1</code>
<code>outform</code>	<code>None str</code>	output format	<code>None</code>
<code>substep</code>	<code>None int</code>	substep	<code>None</code>
<code>substep_reset</code>	<code>bool</code>	if True, the substep will be reset	<code>True</code>
<code>inlimit</code>	<code>None int</code>	input limit	<code>None</code>
<code>outlimit</code>	<code>None int</code>	output limit	<code>None</code>
<code>sep</code>	<code>None str</code>	separator	<code>None</code>

Returns:

Name	Type	Description
<code>bytes</code>	<code>bytes</code>	output of total

```
viewfile(*args, **kwargs)
```

viewfile(arg: str, /) -> pyradiance.rad_params.View

Read a view file into a View object

```
vwrays(pixpos=None, unbuf=False, outform='a', ray_count=1,
pixel_jitter=0, pixel_diameter=0, pixel_aspect=1, xres=512, yres=512,
dimensions=False, view=None, pic=None, zbuf=None)
```

vwrays.

```
write(file_path, data, overwrite=True, mode='wb')
```

Write data to a file.

Parameters:

Name	Type	Description	Default
<code>file_path</code>	<code>str</code> <code>Path</code>	path to file	<i>required</i>
<code>data</code>	<code>str</code> <code>bytes</code>	data to write	<i>required</i>
<code>overwrite</code>	<code>bool</code>	if True, overwrite the file if it already exists	<code>True</code>
<code>mode</code>	<code>str</code>	'w' for text, 'wb' for binary	<code>'wb'</code>

Returns:

Name	Type	Description
<code>str</code>	<code>str</code>	path to file

```
xyz_rgb(*args, **kwargs)
```

xyz_rgb(arg0: float, arg1: float, arg2: float, /) -> tuple